E.C.H.O: An Intent Aware Personal Memory System for Unified Search Across Email, Web, and Video History

1st Arjun Sai N
Dept. of Computer Science and Engineering
PES University
Bengaluru, India

2nd Mohammed Mir Fazlai Ali
Dept. of Computer Science and Engineering
PES University
Bengaluru, India

3rd Mohammed Shehzaad Khan
Dept. of Computer Science and Engineering
PES University
Bengaluru, India

4th Mrigank Jain
Dept. of Computer Science and Engineering
PES University
Bengaluru, India

5th Mannuru Shalima Sulthana
Dept. of CSE (AI&ML)
PES University
Bengaluru, India

***Abstract*—In a single day, a person meets information in many places, such as the inbox, the browser, and video sites. All these places keep their own history behind their own search box, so finding something again starts with guessing where it was seen. We present E.C.H.O (Efficient Contextualized History Orchestrator), a personal memory for a single user. It brings Gmail, Chrome, and YouTube activity into one store on the user's own laptop, and then it answers questions about that activity in plain language. It keeps all mail, except what Gmail itself files as spam or trash. Web pages and videos, though, get kept only if signals such as foreground time, scrolling, watch time, or a return visit show that the person actually paid attention to them. In the background, a worker cleans each item and pulls text out of its attachments. The same worker also puts each item into one of five broad categories and embeds it so that it can be searched by meaning. Whenever the user asks a question, it goes to a language model that can call just one search tool, and this tool is just plain code. The tool runs keyword search and vector search side by side and then merges the two ranked lists using reciprocal rank fusion. If the results look weak, the search gets widened. After that, a local cross encoder reorders what is left. Usually a question costs two model calls, and the code allows at most six. Storage, indexing, and search all run locally. All the model gets is the question and limited excerpts from the top results, so it never sees database rows or the whole store. Apart from this, a wellbeing module describes how time is spent, but it does not judge it. It also sorts items into the user's personal goals. An item gets added on its own only when the module is confident, and all the others wait for the user to approve them. We tested all this on a collection with 119 items and 38 questions. The first ten results held 93% of the relevant items. A question needed 2.3 model calls and took a median of 8.1 seconds. And in all 13 clean scripted sessions, the capture rules matched their specification. The tests show one more thing. On this small collection, semantic search alone ranked the results better than the full pipeline, and we discuss why.**

***Index Terms*—personal information management, retrieval augmented generation, hybrid search, reciprocal rank fusion, tool calling agents, intent aware capture, local first software, digital wellbeing**

I. Introduction

Suppose a student is preparing for a job interview, and the company sends them an email about it. Over the next few days, the student reads articles on operating systems in Chrome and also watches a few tutorials on YouTube. Now a week later, the student wants all that material back. But the email is in Gmail, the videos are in the YouTube watch list, and the articles are buried somewhere in the browser history. These apps all have their own search box, and none of them knows what the other two hold. So the student either has to search all three one by one or start again from scratch.

A machine that remembers what its owner has seen is not a new idea. Bush called it the memex back in 1945 [1]. Later, systems such as Stuff I've Seen [2] and MyLifeBits [3] built personal stores of mail, pages, and documents. But two problems still make such systems hard to use every day. The first one is noise. A browser logs every page that opens, even the stray clicks and the tabs that were closed after two seconds. So if a memory is built directly on top of that log, it fills up with things nobody actually read. The second one is that recall depends on exact words. A search box needs the right keyword, but people remember things loosely, as in "the OS material I read after the interview email."

This is where E.C.H.O helps, as it handles both these problems. It watches Gmail, Chrome, and YouTube, but a page or a video is kept only if the engagement shows that the user really paid attention to it. All the items share one data

model on the user's laptop. There they get enriched in the background, and then the user can ask about them in natural language. Every answer is grounded in the stored items, and this is nothing but the idea of retrieval augmented generation [4].

Overall, our main contributions are:

- An intent aware capture layer that has its own set of rules for each source, along with privacy filters that act before anything gets stored. So sensitive pages lose their text, secret parameters get cut from the addresses, and private work apps are kept with just a page title.
- A local storage model where emails, pages, and videos all share one central table. Here PostgreSQL is the permanent record, Redis holds the short lived state, and the FAISS index can be rebuilt from PostgreSQL. E.C.H.O also extracts the text inside attachments when the mail arrives, so that it can be searched too.
- A bounded agentic retrieval loop, where the language model plans and phrases the searches but a deterministic tool does the actual searching. This tool is responsible for hybrid keyword and vector search, rank fusion, a rule based quality check that widens the search when needed, and cross encoder reranking. Usually a question needs two model calls, and the code puts a hard limit of six on it.
- A wellbeing module that reports time, sessions, and the regret the user declares, but only in descriptive terms. Along with it, there is a grouping engine for personal goals. This engine files an item on its own only when two signals agree, and even then the user can undo it in one click. All the other items have to wait for the user's approval.

Section II goes through the related work, and Section III gives an overview of the entire system. Then Section IV explains capture, storage, enrichment, and retrieval, and Section V describes the wellbeing module along with the user groups. Section VI is about privacy, Section VII is about the implementation, and Section VIII presents the evaluation. Finally, Section IX lists the limitations, and Section X concludes.

II. Related Work

A. Personal Information Management

Bush imagined the memex as a desk that holds a person's books, records, and letters and can bring any of them back quickly [1]. Stuff I've Seen built a single index over everything a person had already seen, such as an email, a web page, or a document. It then used that earlier exposure as a cue for search [2]. MyLifeBits stored personal data in a SQL database, and its aim was to keep everything that could be captured [3]. E.C.H.O has the same goal of refinding, but it differs from them in two ways. First, it filters at the time of capture, so it does not store everything. Second, instead of just giving a query box, it answers questions in natural language.

B. Retrieval Augmented Generation and Tool Use

Retrieval augmented generation pairs a retriever with a text generator, so that the generated answer is based on the retrieved documents [4]. E.C.H.O applies this idea to a personal collection and arranges the steps as a graph built with LangGraph [5]. ReAct showed that a language model can alternate between reasoning and actions such as search calls, and that grounding each step in tool output reduces invented facts [6]. We borrow this pattern, but in a deliberately narrow form. The model has exactly one read only tool. Apart from two small local encoders, this tool has no model inside it. Also, the code caps how many rounds the model gets.

C. Hybrid Search and Reranking

With sentence embeddings such as Sentence BERT, texts can be compared by their meaning using cosine similarity [7]. E.C.H.O embeds every item with a small local sentence model [8] and searches the vectors with FAISS [9]. For keyword search, it directly uses the full text search that comes built into PostgreSQL. The problem is that these two methods score on unrelated scales, so it does not make sense to add their scores directly. Reciprocal rank fusion solves this because it uses just the position of each item in each list [10]. After fusion, a cross encoder reads the question and a candidate together and scores how well they match. Nogueira and Cho showed that this works well for passage reranking [11]. Our cross encoder is a compact MiniLM [12] model, trained on the MS MARCO passage ranking data [13].

D. Local First Software

Kleppmann et al. describe local first software, where users own their data and the application works from a copy on their own device [14]. E.C.H.O follows the same idea, as the database, the vector index, the cache, and both encoders all run on the user's laptop. The hosted language model gets only limited pieces of text.

E. Digital Wellbeing Tools

Lyngs et al. reviewed hundreds of digital self control apps and browser extensions. They found that most of them block, limit, or track usage against a goal [15]. E.C.H.O takes a different approach from these tools. All it does is show where the time went and let the user mark the items they regret. It does not score the user or block any kind of content. It also does not call any time wasted.

III. System Overview

Fig. 1 shows the main parts of the system. First, data comes in through three connectors. The Gmail connector reads mail through the Gmail API with OAuth, and it asks only for the changes since its last check. A Chrome extension keeps watching desktop browsing and YouTube playback, and then the YouTube connector fills in each video's details from the YouTube Data API. All the connectors hand over their items to a single FastAPI gateway, where the storage engine writes each item in one transaction and skips duplicates.

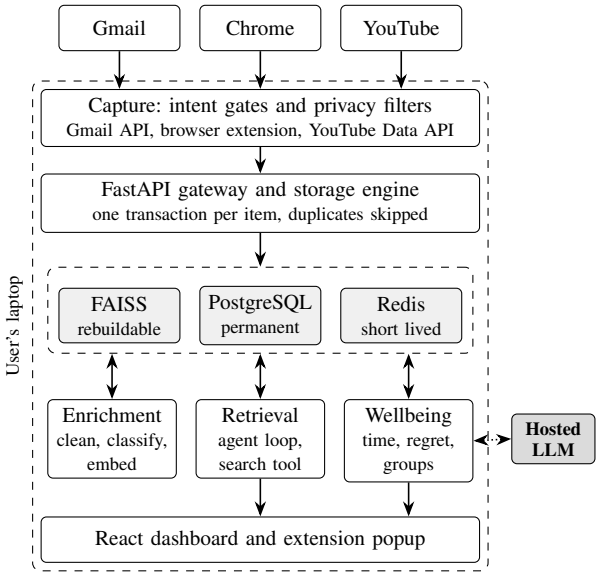


Fig. 1. The main parts of E.C.H.O. Everything inside the dashed frame runs on the user's laptop. The dotted arrow is the only path to the hosted language model of E.C.H.O; it carries bounded text from three callers (classification, retrieval, weekly reflection), described in Section VI.

Next, there are three stores, and each store has its own job. PostgreSQL holds the permanent record of every item. FAISS holds the embedding vectors, and these can be rebuilt from PostgreSQL. Redis is responsible for the short lived state, which covers revisit flags, cached attachment text, reminder state, and the user's capture settings. This entire storage layer runs on the user's laptop itself.

On top of this stored data, three modules do the work. In the background, the enrichment worker gets the new items ready for search, while the retrieval engine is the one that answers questions. The wellbeing module takes the stored activity and turns it into time summaries, sessions, regret patterns, and group suggestions. The user mainly works through a React dashboard, and a small extension popup also lets them pause capture.

IV. Methodology

A. Intent Aware Capture

What should E.C.H.O keep? Each source has its own answer, and Table I summarizes them. These are the values we chose for the final system. The browser and the server hold matching copies, so that the server does not silently refuse a page that the extension sends.

Gmail is treated differently from the other two sources. Mail arrives on its own, so the user does not choose it. Also, few mails arrive each day, and even an unread message may matter later. So we store all messages when they arrive. The connector polls every 30 seconds, but each poll asks Gmail just for the changes since a saved history position. It does not list the entire mailbox each time. If that position has expired, the connector will list the mailbox once. Then the uniqueness rule on stored items absorbs the overlap. We do not read mail that Gmail has already filed as spam or trash, so Gmail's own spam filter also works as the spam filter of E.C.H.O. Whenever the user opens a message in Gmail on the web, the extension reports just the message id and the seconds spent reading.

TABLE I
Capture Rules for Each Source

Source	An item is kept when
Gmail	Always, at arrival, except mail in Spam or Trash. Polled every 30 s for changes only. Reading time is added when a message is opened (opens under 3 s are ignored).
Chrome	Phase 1: at least 5 s in the foreground. Phase 2: any one of at least 10 s in the foreground, at least 15% scroll depth, one interaction other than a link click, or a revisit within 24 hours. Work app pages: at least 5 s or one interaction, title only.
YouTube	Any one of at least 20 s of foreground playback (10 s for Shorts), at least 10% of the video watched with at least 5 s of watching, a pause, seek, or speed change by the user, or a revisit within 24 hours.

Next is Chrome, where a page has to get through a gate that has two phases. In phase one, the page has to stay in the foreground for at least five seconds. If a page fails this phase, we drop it, and we remember only a page that passes it for revisit detection. Phase two comes next, where at least one sign of attention has to appear. This sign can be enough foreground time, enough scrolling, a real interaction, or a visit to the same page within the last 24 hours. Selecting text or clicking inside the page counts as an interaction, but clicking a link does not. This is because a click that leaves the page shows a choice to go elsewhere, and not interest in the page itself. We count time only while the tab is active, the window has focus, and the user has not been idle for 30 seconds. Along with the text of the main content, the extension also sends the top level headings of the page, so that the page can be found later by its topic.

We keep a YouTube video if the user watches it long enough, or watches a large enough share of it. Pausing, seeking, changing the speed, or returning to the same video within a day also keeps it. The completion rule needs at least five seconds of real watching, such that swiping past a short clip does not count as watching ten percent of it. Once a video has been watched for five seconds, the extension records the visit, so that a second short visit can pass through the revisit rule. We identify Shorts directly from the page address, and they get a lower time bar.

B. Storage Model

In storage, each email, page, and video becomes one row in a central table of memory items. The fields which belong to a particular source go into a matching table for Gmail, Chrome, or YouTube, and attachments have a table of their own. Since engagement counters such as reading time, watch time, and revisits change often, we keep them separately. Also, Chrome and YouTube write one row for every visit that gets admitted, along with the visit time and the seconds spent. So if a page

is read on two different days, it counts on both of those days. Overall, the complete schema has 17 tables.

To stop the same item from being stored twice, for example after a connector restarts, there is a uniqueness rule on the pair of source type and source id. Each item always carries one of the five system groups, with misc as the fallback. When an item comes in, we first write it in a raw state and flag it as unprocessed. Doing this keeps the capture path short and safe against crashes. Later, if a revisited page comes back with different text, we flag it again so that its vector gets rebuilt. Timestamps are stored in UTC, and they are shown to the user in local time.

The user can delete a single item, and doing so hides it from all queries. Apart from this, the user can also pick a retention setting to keep items forever, for one year, or for six months. Recalling old things is the whole point of the system, so keeping items forever is the default. If the user picks a shorter window, then a job at backend startup will permanently remove the older items from PostgreSQL, the FAISS index, and Redis together.

C. Background Enrichment

A background worker picks up the unprocessed items every few seconds. The worker runs outside the capture path. So even if an enrichment step is slow, it can never slow down the capture. Then each item goes through four steps.

Cleaning. For email, HTML is converted to text with the line breaks kept, and scripts, styles, tracking pixels, and hidden preview text are removed. When the sending app has split a body around an inline image, the parts are joined back together. Quoted earlier messages are removed, but the lines the sender typed between quoted lines are kept. This is because an inline reply is often the most important part of the message. A heuristic remover that we tested cut real body text in six of eight sample messages, while a leftover signature only adds a little noise. So signatures are deliberately left in place, as a little noise is better than lost body text. For a reply, keywords from earlier messages in the same thread are added, so that a terse "Re:" can still be found by its topic. For web pages, the extension has already picked the main content and the headings, and the worker separates the headings from the body. A video's text is made of its title, channel, category, tags, and description. The server also tries to download the caption text, but this is best effort, and it returned nothing for any of the 14 videos in our test data.

Attachments. The same worker extracts attachment text when the mail is processed. It supports PDF, DOCX, XLSX, PPTX, HTML, forwarded messages, and plain text formats such as TXT, CSV, Markdown, JSON, and calendar files. Images and scanned PDFs are read with Tesseract OCR [16], while inline images such as signature logos, and very small images, are skipped. The extracted text is stored next to the attachment record and not inside the email body. The beginning of each attachment joins the embedded text of the email, while the entire text joins the keyword index. So even the content that sits deep inside a file can still be found by keyword.

Embedding. The `snowflake-arctic-embed-xs` sentence model [8], which reads up to 512 tokens, turns the cleaned text into a 384 dimension vector. To do this, the text is put together such that the short hints come first. For mail, these hints are the subject, keywords, and sender. For pages, they are the domain and headings, and for videos, the channel and category. The long text, such as a body or a description, comes last. The text is then cut to fit the model's 512 token window using its own tokenizer, such that the cut ends at a whole word. So only the end of the long part gets removed, and the text we store is exactly what was embedded. If a mail has attachments, the body takes up to 300 tokens and the attachments fill the remaining space. In a similar way, for a video, the description takes up to 200 tokens and any caption text fills the rest. Finally, the vector is added to the FAISS index without a rebuild.

Classification. Each item gets one of the five broad groups, which are work, study, entertainment, personal, and misc. To choose the group, we use a cascade of four stages. Stage one reads the structural signals that come from the source itself, such as Gmail category labels, academic mail domains, or the YouTube video category. Next, stage two looks up the web domain in a curated table. Then stage three compares the vector of the item with a center point for each group and accepts only a clear winner. At the start, each center is just the mean of several example sentences written for that group. While a center is still a seed, the required lead is smaller, 0.05 instead of 0.08, as we measured on labeled rows (Section VIII). Once a group has at least ten confirmed items, its center is computed again from those real items. Here each source gets an equal weight, so that the center learns the topic and not how one particular source looks. Both kinds of center need a cosine similarity of at least 0.55, and a recomputed center also has to lead the second best group by at least 0.08. This recomputation happens when the worker starts, but only if the stored centers are missing or are older than 30 days. Finally, stage four asks the language model, one item at a time, with only the title and about 300 characters of text. With an OpenAI model, this call is a typed choice among the five group names. With other providers, the reply has to be exactly one group name, and anything else becomes misc. Suppose the typed call raises an error. Then that item takes the single word path instead, so the classification does not stop. We do not store a confidence value.

State tracking. At the end, an item is either processed or failed. The failed ones are kept apart, so that a failure is neither counted as done nor lost. If needed, the worker can try them again through a configuration switch.

D. Retrieval Loop

Next, we look at what happens to a question once the user asks it. It goes through the loop in Fig. 2. The language model we use is GPT-6 Luna. It gets the conversation so far, a small calendar around today's date, the question itself, and one tool

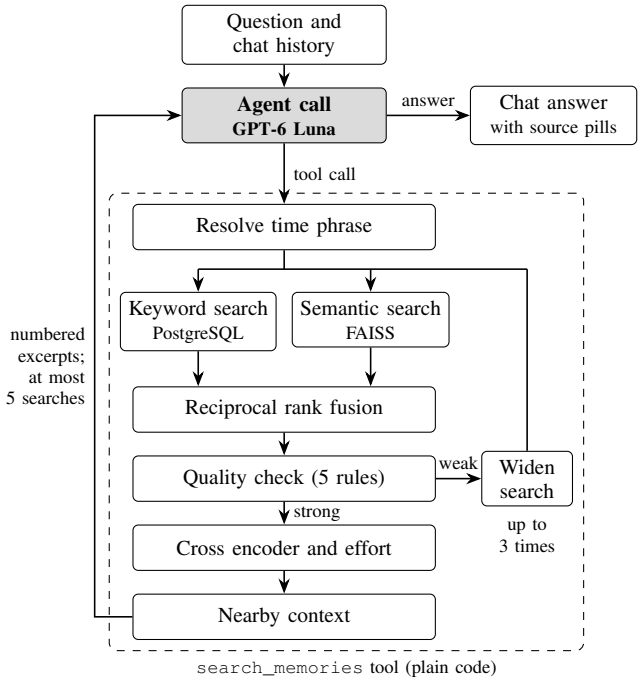


Fig. 2. The retrieval loop. A language model gets called only in the shaded box. Everything inside the dashed frame is deterministic code, along with two small local encoders. When the search is widened, both search branches run again.

called `search_memories`. This tool takes a query made of a few keywords, along with some options. These are up to two alternative phrasings, a list of sources, a start date, a reference item with the word before or after, and a flag to ask for attachment text. The model decides how to phrase the search. Then, in one round, it may issue several searches in parallel, for instance one per source. After reading the results, it either answers directly or searches again. On the first turn of a chat, the first call has to use the tool. A usual question needs one call that searches and one call that answers. The code allows at most five searches for one question, and at most three in one round. After the fifth search, we make the next call without the tool, such that the model has to answer. So a question costs between two and six model calls, and the program enforces this bound, not the prompt.

Resolve time phrase. The model reads the calendar in its first message and passes a start date, so a phrase such as "last Tuesday" reaches the tool as a date. The tool accepts only a valid date. A phrase that points at another item, such as "after the interview email," is passed as a reference item, and the code looks it up among the user's own items. For this lookup, keyword matching is tried first and then meaning, with a minimum similarity. Whichever item matches, its time then becomes a hard limit for both the search branches. If no reference item is found, the search runs without that limit. For mail, the match depends on when it arrived, whereas a page or a video matches if any one of its recorded visits falls in the range. The date filter is open ended, so the answering call is told today's date and asked to use only the items inside the period the user named.

Parallel search. On the keyword side, the branch runs PostgreSQL full text search over a stored text index. This index covers the titles, the complete item text, attachment text, senders, domains, and channel names. On the semantic side, the branch embeds the query and then searches FAISS. Both branches run in the same step of the graph.

Fusion. The two ranked lists are merged with reciprocal rank fusion [10]. For an item d, the fused score is

$$\mathrm{RRF}(d) = \sum_{b \in B} \frac{1}{k + r_b(d)}, \tag{1}$$

where B is the set of branches that returned d, $r_b(d)$ is its position in branch b starting from 1, and $k = 60$. As only the positions matter here, there is no need to match the two score scales at all. The top 50 fused items move on to the next step.

Quality check and widening. Five rules judge the fused list. A list counts as weak or empty when it has no items at all or holds fewer than two items. It is also weak if none of its items come from a source that the user asked for, or if none of them fall inside the time window. The last rule looks at the best cosine similarity, and a list whose best value is below 0.55 is weak too. On our test questions, this floor did not flag a single one of the 38 answerable queries as weak. But it let 2 of the 12 unanswerable queries pass (Section VIII). If a list is weak, the tool will trigger a wider search, and both branches run again. Each attempt relaxes the search more than the one before it. The first attempt stretches the time window by four days, opens up all the sources, or cuts the query down to its core keyword. The second then drops the time filter and the anchor, and the third drops the keyword filters and searches a larger part of the vector index. After three attempts, the tool reports that nothing was found, and the model can either say so or try a different phrasing.

Reranking. The local cross encoder `ms-marco-MiniLM-L-6-v2` [11], [13] scores each remaining item against the question. It reads the title of the item followed by the beginning of its cleaned text, 512 characters in all. The final score blends this with an effort score:

$$s(d) = 0.8\,\sigma\big(c(q,d)\big) + 0.2\,e(d), \tag{2}$$

where $c(q,d)$ is the cross encoder output for question q and item d, σ is the sigmoid function, and $e(d) \in [0,1]$ is the effort score. Effort is the mean of the engagement signals that an item has, such as reading or watch time, scroll depth, revisits, interactions, and how much of a video was watched. Each signal is capped and scaled to $[0,1]$. This favors the items that the user gave real attention to. Email is a special case. If a mail was read on a phone, or never opened in the browser, it has no measurable reading time. Instead of scoring such mail as zero, the reranker gives it the median effort of the other emails in the same result list. This way, an unmeasured email is neither punished nor promoted. By design, the score has no term for how recent an item is, and there is no diversity reranking.

Nearby context and excerpts. For the top three items, the tool also adds their close neighbors. These are other emails in the same thread, or pages and videos from the same browsing session or within five minutes. Each result reaches the model as a numbered entry. The entry carries the item's title, source, and time, along with the sender, site, or channel and the beginning of its cleaned text. This excerpt can be at most 8,000 characters for each of the top three results and 2,000 characters for the rest. Now suppose the question is about what is inside a file attached to an email. In that case the tool also adds the attachment text extracted from the best ranked mail, up to 40,000 characters. These numbers stay the same across all the searches of one question, so that a citation always points to the exact same item.

Writing the answer. The final call writes a conversational reply in Markdown. The reply starts with the direct answer, and if something is missing, it says so plainly. If the question is ambiguous, it asks back in one line. After each fact, the reply puts a hidden marker that names the numbered item supporting it. The backend keeps just those markers that point to items the tool actually returned, so that the model cannot cite a source it did not see. The dashboard then turns each marker into a small pill. The pill names the human anchor of the memory, such as the sender, the site, or the channel, along with a short date. Clicking on it opens the original email, page, or video. Apart from this, a thin time strip under the reply places each cited item on a timeline. While the loop runs, the dashboard shows progress messages, such as which source is being searched, and then it shows the reply.

Failure handling. If the model cannot be reached, the deterministic pipeline runs once on the raw question. In this case, the user still receives the ranked list of matching items, and not an error.

E. Conversation Memory

Each chat has a session id, and we save every turn in PostgreSQL on the laptop. When the user opens an old chat, its session picks up from where it was left. So a follow up such as "show me Chrome pages about that topic after that email" can refer back to an earlier answer. On a new turn, the model gets the last twelve messages of the chat. In these, each question is cut down to 1,000 characters and each answer to 3,000. The citation numbers restart with every question. At startup, we delete all the chat history that is older than 30 days.

V. Wellbeing and User Groups

The whole wellbeing module is built to answer one question, which is where the user's time actually went. It describes how that time is spent, but it does not judge it. So it just shows the numbers and the patterns, and the verdict is left to the user.

Time. We do not store the time totals separately. Instead, they are computed on demand from the stored records, so that the same minutes do not get counted twice. The time for pages and videos comes from the per visit rows, so each day gets exactly the time that was spent on that particular day. For email, the time comes from the reading time that the extension reports. All the days here are the user's local days.

Sessions. We define a session as a run of activity in which consecutive items are less than 20 minutes apart. If the gap were shorter, one sitting would get split into two. For example, suppose the user reads a page for 15 minutes and then opens the next page. As each item is recorded at one point in time, that read looks like a 15 minute gap. For each session, the module keeps track of how long it was, the system group that took the most time, and how many times the user switched between sources.

Regret. What counts as regrettable is decided by the user alone, and any item can be marked. Each mark or unmark gets stored as a new event, so the record does not get overwritten. An item is regretted at the moment if its count of events is odd. Then the module reports how many marked items fall in each content type and each hour. It also flags a habit whenever the same kind of item, at a similar hour, is marked at least three times in 30 days.

Weekly reflection. Once a week, the module can write a short reflection. For this, one language model call receives only aggregated numbers, which are totals per category, regret counts, session patterns, and peak hours. It does not receive a title, an address, or any text from an item. Before the reply is shown, it is checked for judging phrases. If it fails this check, or if the call itself fails, a simple summary built from the same numbers is shown instead.

User groups. System groups answer "what kind of item is this?", while user groups answer "which of my goals does it serve?", for example "Placement prep." A user group starts with the items that the user files by hand. Once it has five confirmed items, an assignment job scores other items against it in two ways. The first way uses the rules the user wrote, each on a keyword, domain, channel, sender, or date range. The second way is a nearest neighbor score, which is the mean cosine similarity between the item and the three most similar confirmed members. An item is suggested when a rule matches, or when the neighbor score alone reaches a threshold. We set this threshold to 0.85, the lowest value at which no item outside a group scored at or above it in our test (Section VIII). The module can also propose a new rule when at least five confirmed members share a domain, sender, or channel.

When does E.C.H.O add something on its own? It does so only for a confident suggestion. A suggestion is confident if a rule matches and the neighbor score also reaches that same threshold. We add such an item directly, but the item still appears in the weekly review, where one click undoes it. So we handle confident items with a human on the loop. Anything less certain waits for an explicit accept in the review screen, which means the human stays in the loop. Also, an automatic addition does not count as a confirmed member until the user keeps it. So one wrong addition cannot pull in the next similar item. Manual filing and automatic additions pass through the same recorded path, so each membership has an audit trail. We never offer a rejected or removed suggestion again for that group. User groups are independent of the five system

groups and have no parent among them, such that one goal can hold study pages and work mail together.

VI. Privacy by Design

What is never captured. Nothing from incognito windows is recorded, and the same goes for typed text and form input. Some pages also keep no text at all. A page falls in this case when its address contains a sensitive word, such as login, password, checkout, otp, or netbanking. Each word is matched as a whole word against a fixed list of 38. A page with a visible password box, or with a form of several personal fields, is treated the same way. Parameters that often carry secrets, such as tokens, one time codes, and session ids, are cut from every address before it is stored. Seven work apps, which are Slack, Atlassian, Notion, Figma, Linear, Trello, and Asana, are stored only as page titles and not as text. Their public help, blog, and documentation sections are still treated as ordinary pages. Gmail on the web is skipped by browser capture, because the mail already arrives through the Gmail API. All these lists live in one copy on the server, and the extension fetches them at startup. Until they arrive, the extension extracts no text.

What leaves the laptop. The database, the vector index, the cache, and both encoders run locally. Two kinds of traffic leave the machine. First, requests to the Gmail and YouTube APIs fetch the user's own data. Second, calls to the hosted language model carry text from three callers. Retrieval sends the question, the recent chat history, and the numbered results. Each result is a title, a time, a sender, site, or channel, and the beginning of its text, at most 8,000 characters for each of the top three results and 2,000 for the others. When the user asks about a file, the text of one attachment is added, at most 40,000 characters. The classifier sends a title and about 300 characters, and it does so only for the items that the first three stages could not place. The weekly reflection sends only aggregated numbers. So a short email can reach the model whole, but a longer email, a page, or a transcript never does beyond these limits, and database rows and vectors are never sent. Requests to the hosted service are also subject to the data retention policy of its provider. Also, when the user connects an MCP client (Section VII), the tool results go to that client, so its own language model sees the same excerpts.

Captured text is treated as data. Suppose a stored email or page contains text that was written to steer a language model. To handle this, the search tool is read only, and the system prompt tells the model that every excerpt is just data. Together, these limit what such text can do. The retrieval logs record counts, scores, and timings, but they do not record questions or titles.

User control. The user can switch off each source, exclude domains and senders, and pause capture from the extension popup. Items can be deleted as well, and the user's entire data can be exported as JSON or CSV. Also, the retention setting removes old data permanently.

TABLE II
Technology Used in the Final System

Layer	Technology
Capture	Chrome extension (Manifest V3), Gmail API, YouTube Data API
Backend	Python, FastAPI, server sent events
Storage	PostgreSQL 16, FAISS, Redis, all local
Enrichment	Text extraction for PDF, DOCX, XLSX, PPTX, and EML; Tesseract OCR
Models	`snowflake-arctic-embed-xs` embeddings (384 dimensions, 512 tokens), `ms-marco-MiniLM-L-6-v2` cross encoder, GPT-6 Luna through the OpenAI Responses API
Retrieval	LangGraph, LangChain, MCP Python SDK (stdio)
Interface	React, TypeScript, Vite

VII. Implementation

All the technology we used is listed in Table II. The backend is written in Python, and FastAPI serves it. For the retrieval loop, we built a LangGraph graph [5] where an agent node and a tool node take turns under a round counter. Every model call goes through LangChain, so moving to a different provider is just a matter of changing one configuration entry. If the provider is OpenAI, the code also turns on the features that are specific to it. These features are the Responses API, a medium reasoning effort for both the retrieval calls, and the typed choice that the classifier uses. The embeddings and the cross encoder run locally on the CPU through the sentence transformers library. We built the dashboard with React and TypeScript, and the extension uses Chrome Manifest V3.

PostgreSQL 16 runs as a native service on the laptop, and Redis runs locally under the Windows Subsystem for Linux. So both stay on the laptop, and only standard HTTPS traffic leaves the machine.

E.C.H.O also runs as a server for the Model Context Protocol (MCP) [17]. Over the stdio transport, it exposes two read only tools, `search_memories` and `get_memory`. The first one has the same contract as the tool in the retrieval loop. So any MCP client, such as Claude Desktop, can query the memory with its own language model. In that case, the client plans the searches instead of the agent loop, while E.C.H.O stays the engine that does the actual searching.

VIII. Evaluation

A. Setup

We evaluated E.C.H.O on just one test account, which belongs to the authors. It holds 119 items, which are 78 emails, 30 pages, and 11 videos. Five of the emails carry a PDF attachment, and we extracted its text. The authors wrote 38 questions of five kinds. There are 16 about one item in one source, 4 about several sources, 6 with a time reference ("emails I received today," "after the interview email"), 6 follow up questions that refer to an earlier turn, and 6 about the content of an attachment. Before the run, the authors marked the relevant items for each question. There are 81 of them in all, with at most eight for one question. We ran everything on

TABLE III
RETRIEVAL QUALITY ON 38 QUESTIONS

Configuration	R@10	R@5	MRR
Keyword only	0.697	0.697	0.693
Semantic only	0.955	0.934	0.987
Fusion, no cross encoder	0.955	0.934	0.987
Full pipeline	0.927	0.905	0.899
Full agent loop	0.927	0.905	0.899

one Windows laptop and called the hosted model through its API.

B. Retrieval Quality

We compare five configurations. The first four replay, for each question, the arguments that the model chose in its first search, so that they differ only in the stages of the tool. These are keyword search alone, semantic search alone, fusion without the cross encoder, and the full pipeline with quality check, widening, and reranking. The fifth is the whole agent loop. Table III gives Recall@10, Recall@5, and the mean reciprocal rank (MRR) of the first relevant item within the first ten results. Recall@k counts the relevant items among the first k results and divides that by the smaller of k and the number of relevant items.

Keyword search alone reaches a recall of 0.70, because the full text search needs every query word to match, and for 11 of the 38 questions it found no relevant item. Semantic search alone and its fusion with keyword search both reach 0.955 with an MRR of 0.987. So on this collection, the keyword branch did not change the first ten results. Now the full pipeline scores lower. For five questions, the right item moved from first place to second or third place after reranking. In the two questions that we examined in detail, the cross encoder gave scores below 0.01 to all candidates. So the effort term of Eq. (2) decided the order, and an email that the user had opened outranked the right one. We therefore cannot claim that reranking helps on small collections of this kind.

The agent loop and the full pipeline have the same top ten, because later searches add items after the first ten. The loop pays off when we count everything it showed to the model. That was 70 of the 81 relevant items, against 65 for a single pass (0.86 against 0.80), at the cost of showing more results. The gain came from three questions in which a second search found items that the first one missed. If we look at each kind of question separately, the loop reaches a recall of 1.00 for single source, follow up, and attachment questions. For questions over several sources it reaches 0.74, and for questions with a time reference it reaches 0.71. In the question “after the interview email,” the model wrote the phrase into the query and did not use the reference item, which gave a recall of 0.20. The answers to all 11 questions whose answer contains a unique fact were correct.

C. Cost and Speed

The 38 questions made 44 turns, since each follow up has two. A turn needed 2.34 model calls on average. In detail, 33 turns used two calls, 8 used three, 2 used four, and 1 used five. None of the turns used up all five searches, and the largest number that a turn used was four. The median response time was 8.1 seconds, the 95th percentile was 23.8 seconds, and the longest was 30.2 seconds. On average, a turn sent 8,983 input tokens and received 238 output tokens. At an assumed price of 0.20 and 1.20 US dollars per million input and output tokens, this comes to about 0.002 dollars per turn.

D. Capture Accuracy

We replayed 15 scripted browsing and viewing sessions in a test profile of Chrome with the extension. The sessions include a page closed after three seconds, a page read for six seconds without scrolling, a page scrolled to 20 percent, a click on a link, and a text selection. They also include a page address with a secret parameter, a video closed after three seconds, a ten minute video watched for seven seconds twice, and a video watched for 25 seconds. Before the run, we wrote down the expected result of each session from the rules. Thirteen sessions were clean, and in all 13 the stored items matched the rules (8 Chrome and 5 YouTube sessions). Two further sessions were discarded because a person used the browser during them. A script drove the sessions and also kept the system from going idle, and the durations are accurate to about two seconds.

E. Privacy Audit

During the evaluation, we recorded all requests to the language model. The largest request held 38,469 characters. Two items in the collection are longer than the 8,000 character limit for the top results, and both were shown to the model. The text beyond the limit appeared in no request.

F. Threshold Calibration

Three similarity cutoffs depend on the embedding model, so we measured them on the collection. First, we looked at the seed centers of classification stage three. For the 78 emails and pages whose group is known, nothing fires at a lead of 0.08. At 0.05 there are 14 firings, and all of them are correct. But no item scored below 0.585, so we could not test the score floor of 0.55. Second, we looked at the quality check. Here the best cosine similarity of the 38 answerable queries was at least 0.558, with a median of 0.734. The 12 queries that have no answer reached at most 0.573. So a floor of 0.55 flags none of the good queries, and it lets 2 of the 12 bad ones pass. If we raised it to 0.58, we would just trade one query for another, so we kept it at 0.55. Third, we looked at the nearest neighbor score of the user groups. For this, we formed three groups of 13, 5, and 5 items. We then scored each member against the other members of its group, and every other item against the group. At 0.65, all 23 members pass, but so do 155 of the 358 other items. At 0.85, 7 members pass and not a single other item does. This is the lowest value at which no wrong item gets in, so this is the value we use. Also, the authors wrote the groups and their labels themselves, and the sample is small.

IX. Limitations and Future Work

E.C.H.O does have some limitations. Currently, it targets just desktop Chrome, so it cannot see what the user does on the phone. The YouTube history starts only from the day the extension is installed, and E.C.H.O does not track playback in the background or in picture in picture mode. Each item has just one vector, so for a long document only the first 512 tokens shape its meaning based match. The deeper content can be reached only by keyword. Replies from Outlook quote the older thread without a marker that E.C.H.O removes, so such threads can show up more than once in the results. The date filter is open ended, and only the top ten results of each search reach the answering call. So on a busy day, a question about yesterday can get crowded out by the items from today. E.C.H.O captures pages from private apps outside the seven listed, such as chat assistants or online documents, just like ordinary pages, unless the user excludes their domain. We keep the OAuth tokens as plain files on the laptop, and the only thing protecting them is the file permissions of the operating system. We download video captions from a public endpoint on a best effort basis. For our videos none of them came back, so a video's text depends on its title, tags, and description. Scanned files need the Tesseract program, which was not installed in our test setup, so we could not evaluate them. Our evaluation covers just one account, with 119 items and 38 questions that the authors wrote themselves. It does not cover the wellbeing module or the group suggestions, which were not tested with users. A single question moves a recall figure by about 2.6 percentage points, so the results may not carry over to larger collections or to other users. Finally, the language model is a hosted service, so its availability and its limits also apply.

Our future work starts with a larger evaluation over several users, where the questions are written by people who did not build the system. Apart from that, we plan to store several vectors for every long item and to let the domain table learn from the labels that the user has confirmed. We also plan to revisit the reranking stage, because in our test the weight on engagement outweighed relevance. Finally, if we import past history from Google Takeout and from the browser's own history file, a new user could start with a complete memory instead of an empty one.

X. Conclusion

Overall, E.C.H.O brings email, browsing, and video history together into one personal memory, and the user can search it in plain language. It keeps all the legitimate mail, but from pages and videos, it keeps only those that the user actually paid attention to. It stores all these items on the user's laptop and gets them ready for search in the background. A question is answered by a loop. In it, the language model plans the search and deterministic code carries it out. Because of this, we can clearly see what the model does, its role stays limited, and its cost stays low. Apart from retrieval, we also have a wellbeing module. It describes how time is spent and keeps the user in charge of every personal grouping. On a test collection of 119 items, a question needed 2.3 model calls and a median of 8.1 seconds. The first ten results held 93 percent of the relevant items. But the tests showed us where the design falls short. The reranking stage did not improve on semantic search, and the model did not always make use of a reference to another item.

Acknowledgment

We would like to thank the Department of Computer Science and Engineering, PES University, for supporting us throughout this capstone project.

References

[1] V. Bush, "As we may think," *The Atlantic Monthly*, vol. 176, no. 1, pp. 101–108, Jul. 1945.

[2] S. Dumais, E. Cutrell, J. J. Cadiz, G. Jancke, R. Sarin, and D. C. Robbins, "Stuff I've seen: A system for personal information retrieval and re-use," in *Proc. 26th Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Toronto, Canada, 2003, pp. 72–79.

[3] J. Gemmell, G. Bell, and R. Lueder, "MyLifeBits: A personal database for everything," *Commun. ACM*, vol. 49, no. 1, pp. 88–95, Jan. 2006.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[5] LangChain, "LangGraph," GitHub repository. [Online]. Available: https://github.com/langchain-ai/langgraph (accessed Oct. 21, 2026).

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992.

[8] L. Merrick, D. Xu, G. Nuti, and D. Campos, "Arctic-Embed: Scalable, efficient, and accurate text embedding models," arXiv:2405.05374, 2024.

[9] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.

[10] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Boston, MA, USA, 2009, pp. 758–759.

[11] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[12] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 5776–5788.

[13] P. Bajaj *et al.*, "MS MARCO: A human generated machine reading comprehension dataset," arXiv:1611.09268, 2016.

[14] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178.

[15] U. Lyngs *et al.*, "Self-control in cyberspace: Applying dual systems theory to a review of digital self-control tools," in *Proc. CHI Conf. Human Factors in Computing Systems (CHI '19)*, Glasgow, UK, 2019, doi: 10.1145/3290605.3300361.

[16] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

[17] Model Context Protocol, "Model Context Protocol specification." [Online]. Available: https://modelcontextprotocol.io/ (accessed Oct. 26, 2026).